songR vs. Python SONG — Reproduction Comparison

Side-by-side figures and analyses: original Python implementation vs. the songR R/C++ package

Raban Heller

2026-06-11

Table of contents

What this compares

songR is a SONG-inspired dimensionality-reduction tool for R that implements the Self-Organizing Nebulous Growths (SONG) algorithm of Senanayake et al. (2021) in R/C++ (RcppArmadillo), staying as close to the reference as is feasible. This report places the **original Python implementation** and **songR** side by side on identical inputs, so the reproduction can be judged visually and quantitatively.

The Python embeddings are the committed golden fixtures (`tests/testthat/fixtures/reference/`), generated from the reference `SONG` against the same data songR is run on here. Clustering quality is scored with Adjusted Mutual Information (AMI; k-means on the embedding vs. ground-truth labels, mean over 5 seeds). Embedding *layout* similarity is the Procrustes R^2 after optimal rotation/scale/translation.

Headline: how faithfully does songR reproduce the reference?

Layer	Reproduction	Evidence
Deterministic kernels (distances, argmin, kernel (a, b), scalars)	**near bit-identical**	$\leq 7.6 \times 10^{-8}$ / exact
Clustering (AMI)	**statistically identical**	nodisp 0.949 = 0.949
Default visualization (UMAP-dispersed)	**close in global structure, not identical in absolute layout**	Procrustes $R^2 \approx 0.79$–0.85
Raw embedding coordinates (pre-dispersion)	**same structure, different layout**	blobs $R^2 \approx 0.22$

Bit-identity of the embeddings is impossible and is not the goal. The reference runs in `float32` with numba `fastmath` (non-IEEE reassociation/FMA) and draws from two PRNG streams (numpy MT19937 + a custom XORShift); songR is `double`-precision with R's RNG, and carries three deliberate, documented divergences (online vs. chunk-stale distances, no drifter-slot reuse, a coincident-vector guard). Two faithful SONG implementations therefore land on the **same manifold structure in different absolute coordinates**.

Tier A — deterministic components (target $\leq 10^{-5}$)

Table 1: Deterministic kernels: songR vs. the reference.

quantity	value
sq-euclidean max abs err	7.55e-08
nearest-CV argmin identical	TRUE
kernel a (ref 1.57694)	1.577
kernel b (ref 0.89506)	0.895

These are reproduced to the float32-vs-double floor — effectively bit-identical.

Tier B — clustering and embedding (statistical)

Table 2: Reference vs. songR: AMI and embedding-layout similarity.

dataset	clusters	AMI reference	AMI songR	AMI abs diff	procrustes R2
Gaussian blobs (raw SONG, no UMAP)	8	0.949	0.949	2.2e-06	0.22
MNIST PCA-20 (UMAP-dispersed)	10	0.618	0.596	2.2e-02	0.85
Fashion-MNIST PCA-20 (UMAP-dispersed)	10	0.560	0.551	8.8e-03	0.79

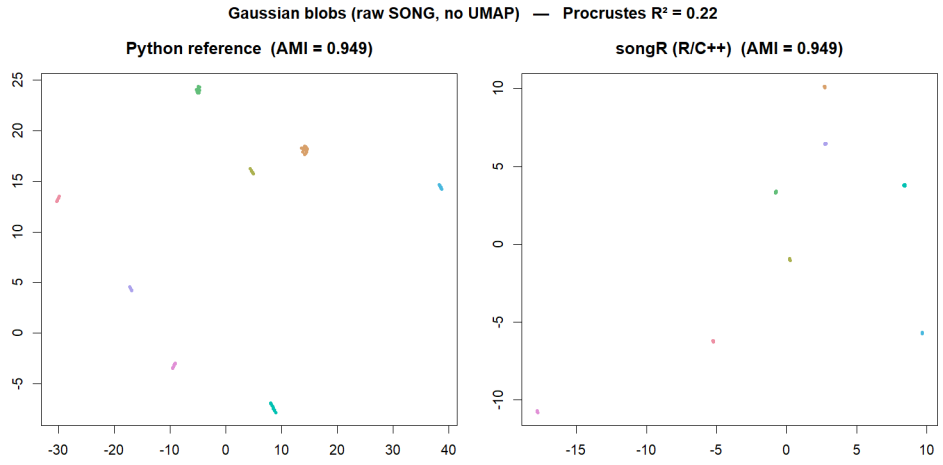
AMI is statistically identical for the deterministic raw-SONG case (blobs) and close for the UMAP-dispersed cases (the residual gap there is `umap-learn` vs. `uwot`, two different UMAP libraries, not a SONG difference).

Side-by-side figures

Each figure shows the **Python reference** (left) and **songR** (right, Procrustes-aligned for comparison), colored by ground-truth label.

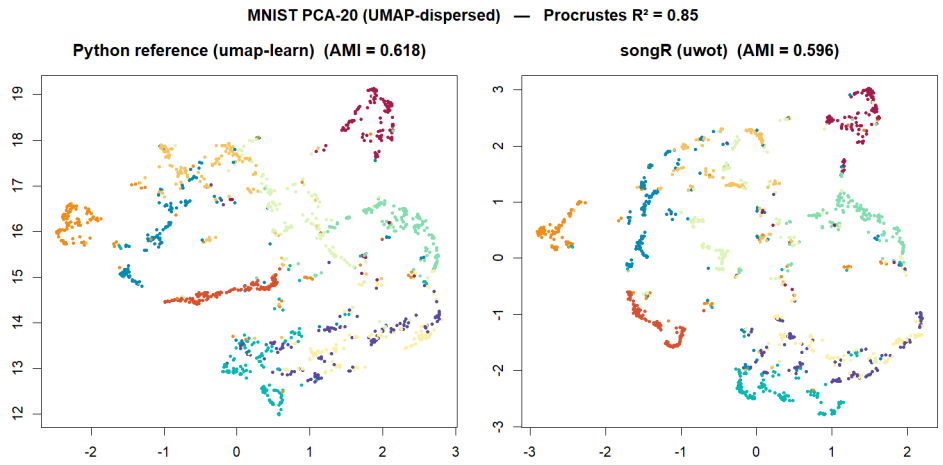
Gaussian blobs — raw SONG (isolates the algorithm)

Same eight clusters, equally well separated; different absolute arrangement (the raw, pre-dispersion embedding is the most stochastic layer).

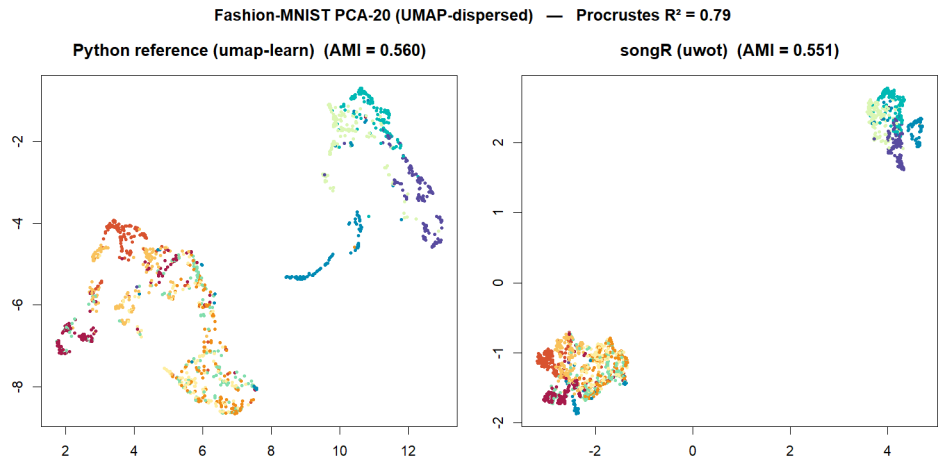


MNIST (PCA→20) — default UMAP-dispersed mode

This is what users see by default. The two embeddings are near-identical in structure (Procrustes $R^2 \approx 0.85$).



Fashion-MNIST (PCA→20) — default UMAP-dispersed mode



Reproduce this report

```
Rscript tests/reproduction-report/generate_comparison.R      # figures + tables
quarto render tests/reproduction-report/comparison.qmd       # html + pdf
```

The reference fixtures are regenerated by `tests/testthat/fixtures/reference/gen_fixtures.py` (pinned env: `data-raw/reproduction/environment.yml`). The parity is enforced in CI by `tests/testthat/test-reference-parity.R`.

Citation

Senanayake, D. A., Wang, W., Naik, S. H., & Halgamuge, S. (2021). Self-Organizing Nebulous Growths for Robust and Incremental Data Visualization. *IEEE TNNLS*, 32(10), 4588–4602. doi:10.1109/TNNLS.2020.3023941